Hand-digit Recognition from Scratch NOTES

Transform

```
transforms.ToTensor()
```

👉 Converts:

- 0–255 → 0.0–1.0
- numpy → tensor

Dataset Loading

```
datasets.MNIST(
    root="data",
    train=True,
    download=True,
    transform=transform
)
```

Meaning:

Parameter	Meaning
root	where data is stored
train	train or test split
download	auto download
transform	preprocessing

DataLoader (VERY IMPORTANT)

```
DataLoader(dataset, batch_size=32, shuffle=True)
```

👉 Converts dataset → batches

Instead of:

- 60,000 images at once ❌

You get:

- 32 images at a time ✅

---

1. Deep Learning Concepts

Tokenizers vs. Embeddings

- **Tokenizer:** The "Dictionary." Breaks text into pieces and assigns meaningless ID numbers (e.g., "I" = 1045). No math can be done on these numbers.
- **Embedding:** The "Meaning." Converts ID numbers into a dense mathematical vector (e.g., a 384-dimensional list of decimals). Enables vector math (Cosine Similarity) so the computer understands that "Apple" is closer to "Orange" than to "Laptop."

Mini-Batches vs. Full Batch Training

- **Full Batch:** Passing the entire dataset (e.g., 60,000 images) into the model at once. Causes immediate Out-Of-Memory (OOM) crashes.
- **Mini-Batching (`DataLoader`):** Feeding data in bite-sized chunks (e.g., `BATCH_SIZE = 128`).
- **The Math Change:** Because the model updates 100+ times per epoch, you must accumulate loss (`train_loss += loss.item()`) and divide by the number of batches (`len(dataloader)`) to find the epoch's average loss.

---

2. The Professional PyTorch Workflow

The industry standard 6-step framework for building neural networks.

Step 1: Data Preparation & The "Three Vaults"

To prevent data leakage, data must be strictly divided into three vaults:

1. **Training Set (80%):** The Gym. The model actively learns and updates weights here.
2. **Validation Set (20%):** The Practice Exam. Evaluated at the end of every epoch to tune hyperparameters and check for overfitting. The model *never* learns from this.
3. **Testing Set (Locked):** The Final Exam. Touched only **once** at the very end of the project to generate the final classification report.

Optimization: Mathematical normalization centers data around zero for faster learning. We used MNIST's exact global mean and standard deviation:

Python

```
transforms.Normalize((0.1307,), (0.3081,))
```

Step 2: Model Architecture

Used a **Convolutional Neural Network (CNN)** (TinyVGG style) built with `nn.Sequential` for clean, block-based routing.

- *Conv2d Layers:* Extract features (edges, curves). `padding=1` preserves spatial dimensions.
- *MaxPool2d:* Compresses image dimensions by half, saving compute.
- *Hidden Units:* Increased from tutorial-level (10) to production-level (32) to handle messy, real-world human handwriting.

Step 3: Rules of Learning

- **Loss Function:** `nn.CrossEntropyLoss()` (Standard for multi-class classification).

- **Optimizer:** Upgraded from `SGD` to `Adam` (`lr=0.001`). Adam automatically adapts its learning rate, finding the bottom of the loss curve faster and smoother.

Step 4 & 5: The Train/Validation Loop

The strict 5-step calculus engine inside the training loop:

1. **Forward Pass:** `y_pred = model(X)`
2. **Calculate Loss:** `loss = loss_fn(y_pred, y)`
3. **Zero Gradients:** `optimizer.zero_grad()` (Clears memory from the last batch).
4. **Backpropagation:** `loss.backward()` (Calculates the errors of specific weights).
5. **Optimizer Step:** `optimizer.step()` (Updates the weights).

Step 6: Artifact Export

- Never deploy the training environment.
- Export strictly the optimized weights (`state_dict`) using `torch.save()`. This turns a massive notebook into a ~1MB `.pth` file.

---

3. Professional Evaluation Metrics

Loss Curves (Detecting Overfitting)

- **Healthy:** Both Training Loss and Validation Loss slope downward together.
- **Overfitting:** Training Loss goes down, but Validation Loss spikes upward. This means the model is memorizing the training images instead of learning general shapes.

The Confusion Matrix

A heatmap grid comparing *Predicted* labels vs. *True* labels. Reveals exactly where the model is failing visually (e.g., constantly mistaking the digit '4' for '9').

Precision & Recall

For multi-class classification (0-9), ROC/AUC are less useful.

- **Precision:** When the model guessed '5', how often was it actually a '5'? (Avoids False Positives).
- **Recall:** Out of all the real '5's in the dataset, how many did the model find? (Avoids False Negatives).

Inference Compute Optimization

During evaluation, raw logits are outputted. Avoid running `torch.softmax()` just to get predictions. `softmax` is mathematically expensive. Since the highest logit will always become the highest percentage, simply use `y_logit.argmax(dim=1)` to save compute.